DESIGN PATTERNS · STRUCTURAL | GoF · 08 / 23 | 4-dive study card

Composite

Compose objects into part-whole trees, and let clients treat one object and a whole group uniformly.

Airplane porthole — DIVE 01 · ORIENTATION

▼ 10 km

Composite builds part-whole trees where a group implements the same interface as a leaf — so clients treat one object and many alike.

— THE CONTOURS

- **Treat leaf and group alike** — one interface for a single item and a whole tree
- **Recursive part-whole tree** — composites contain leaves and other composites
- **Operations cascade** — call the root; it recurses to every child

— USE IT WHEN — GoF applicability

- **Part-whole hierarchies** — you want to represent objects as trees of part-whole structures
- **Uniform treatment** — clients should ignore the difference between a leaf and a composite

Structural family — Composite makes a part-whole tree uniform; Decorator is a degenerate composite with one child that adds behavior; Iterator and Visitor walk the tree it builds.

DIVE DEEPER

Train window — DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

- **Component** — common interface for both leaf and composite (operation())
- **Leaf** — a primitive with no children — does the actual work
- **Composite** — holds children, implements operation() by recursing

— IN THE WILD

- **DOM nodes** — element & text nodes share the Node interface
- **java.awt Container** — Containers hold Components — and are Components
- **React component tree** — parents and leaves render through one API
- **File system** — GoF's example — folders contain files & folders

`<>` Client → Component.operation() → Composite recurses into each child Leaf or Composite.

GoF intent: "Compose objects into tree structures to represent part-whole hierarchies."

DIVE DEEPER

Park walk — DIVE 03 · CONNECTIONS

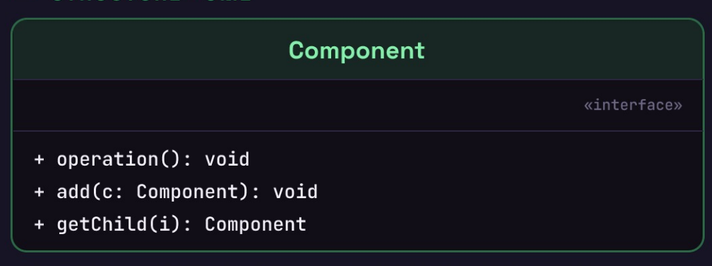
▼ 100 m

Client (holds a Component) → operation() (one call) → Composite recurses (visit each child) → ✓ leaves do the work (results combine)

— WHY IT FOLLOWS

- → **Shared Component interface ⇒** a leaf and a whole subtree are invoked exactly the same way
- → **Composite holds children ⇒** it implements operation() by delegating to each child
- → **Recursion bottoms out ⇒** leaves do the real work; composites just aggregate results

— STRUCTURE · UML



Leaf implements operation(); Composite stores children and implements operation() by calling it on each. Child ops may live only on Composite (safety vs transparency).

— WHAT IT BUYS YOU

- ✓ **Uniformity for clients —** client code ignores the leaf-vs-composite distinction
- ✓ **Easy to add components —** new leaves or composites fit the existing interface
- ✓ **Natural recursion —** tree operations become simple recursive delegations
- ✓ **Open hierarchies —** arbitrarily deep and arbitrarily wide structures
- ✓ **Generality trade-off —** a uniform interface can make a tree overly permissive

Decorator is a Composite with a single child that adds behavior; Iterator traverses it; Visitor applies one operation across the whole tree; a Builder can construct it.

DIVE DEEPER

Microscope — DIVE 04 · DETAILS

▼ 1 cm

Definition: "Compose objects into tree structures to represent part-whole hierarchies; Composite lets clients treat individual objects and compositions of objects uniformly."
— GoF, "Design Patterns", 1994

— THE CODE

```
interface Component { size(): number; }
class File implements Component {  // Leaf
constructor(private bytes = 0) {}
size() { return this.bytes; }
}
class Folder implements Component {
kids: Component[] = [];   // Composite
size() {  // recurse, then aggregate
return sum(this.kids.map(k => k.size()));
}
}
```

— VARIANTS

- **Transparent** — add/remove on Component — uniform, unsafe on a leaf
- **Safe** — child ops only on Composite — type-safe, less uniform
- **With parent refs** — each child points back up for easy traversal
- **Shared leaves** — flyweight leaves shared across many trees
- **Cached aggregate** — composites cache totals, invalidate on change
- **Ordered children** — a list keeps child order when it matters

— COMPARE THE ALTERNATIVES

Aspect	Composite	Decorator	Iterator	Visitor
Builds a tree	✓	~	×	×
Uniform leaf / group	✓	✓	×	×
Adds new behavior	×	✓	×	✓
Traverses structure	~	×	✓	✓
Recursive by nature	✓	✓	~	✓

Best fit → Composite: part-whole trees · Decorator: layer behavior · Iterator: walk elements · Visitor: apply ops across a tree. ("~" = partial.)

— NUANCES & GOTCHAS

- **Transparency vs safety** — put add/remove on Component (uniform) or only on Composite (type-safe)? GoF leans transparent
- **Leaves must handle child ops** — in the transparent form, add()/remove() on a Leaf should no-op or throw
- **vs Decorator** — a Decorator is just a Composite with exactly one child that adds behavior
- **Parent references** — ease traversal and removal, but add coupling and cycles to manage
- **Recursion cost** — deep trees risk stack depth; an explicit stack or Iterator helps
- **Don't over-generalize** — a uniform interface can become a grab-bag of rarely-valid methods

— INTERVIEW PITFALLS

Q "Transparency vs safety?"
Transparent puts add/remove on Component so all nodes look alike (leaves may fail them); safe puts them only on Composite (type-safe, less uniform). GoF favors transparency.

Q "Composite vs Decorator?"
Same recursive structure, but a Decorator is a Composite with a single child whose job is to add behavior, not to aggregate many children.

Q "How do leaves handle add()?"
In the transparent design they implement it as a no-op or throw, since a leaf has no children to add to.

Q "Where do you see it?"
The DOM, UI component trees, file systems, menus, org charts — anything part-whole where a node and a subtree share an interface.

— WATCH OUT

⚠ **Anti-pattern:** Skip it without a real part-whole tree — a flat list or plain object graph is simpler. Forcing leaves to expose child operations they can't honor trades clarity for false uniformity.

⚡ **Modern take:** you use Composite constantly without naming it — the DOM, React / Vue trees and file systems are all composites; reach for it whenever an operation must hit one node or a whole subtree the same way.